(`head` ❶ and `tail` ❷), you could in principle use two mutexes, one to protect `head` and one to protect `tail`, but there are a couple of problems with that.

The most obvious problem is that `push()` can modify both `head` ❺ and `tail` ❻, so it would have to lock both mutexes. This isn't too much of a problem, although it's unfortunate, because locking both mutexes would be possible. The critical problem is that both `push()` and `pop()` access the `next` pointer of a node: `push()` updates `tail->next` ❹, and `try_pop()` reads `head->next` ❸. If there's a single item in the queue, then `head==tail`, so both `head->next` and `tail->next` are the same object, which therefore requires protection. Because you can't tell if it's the same object without reading both `head` and `tail`, you now have to lock the same mutex in both `push()` and `try_pop()`, so you're no better off than before. Is there a way out of this dilemma?

Enabling Concurrency by Separating Data

You can solve this problem by preallocating a dummy node with no data to ensure that there's always at least one node in the queue to separate the node being accessed at the head from that being accessed at the tail. For an empty queue, `head` and `tail` now both point to the dummy node rather than being `NULL`. This is fine, because `try_pop()` doesn't access `head->next` if the queue is empty. If you add a node to the queue (so there's one real node), then `head` and `tail` now point to separate nodes, so there's no race on `head->next` and `tail->next`. The downside is that you have to add an extra level of indirection to store the data by pointer in order to allow the dummy nodes. The following listing shows how the implementation looks now.

Listing 6.5. A simple queue with a dummy node